

Higher-Order Functions

```
map([🐮, 🍌, 🐔, 🌽], cook)  
=> [🍔, 🍟, 🍗, 🍿]
```

```
filter([🍔, 🍟, 🍗, 🍿], isVegetarian)  
=> [🍟, 🍿]
```

```
reduce([🍔, 🍟, 🍗, 🍿], eat)  
=> 🤩
```

Wiederholung: Lambda



Ein Lambda ist eine **anyonyme Methode**, die **lazy evaluiert** wird, d. h. sie wird **erst implementiert** und **zu einem späteren Zeitpunkt ausgeführt** (evaluiert).

```
@FunctionalInterface
public interface MoneyPrinter {
    void amount(int value);
}
```

```
public static void main(String[] args) {
```

```
    MoneyPrinter moneyPrinter = (int value) -> System.out.print("€" + value);
```

```
    moneyPrinter.amount(42);
```

```
}
```

Evaluation

↑
Typ kann weggelassen werden,
da dieser von Java inferierbar ist.

Implementation

Ausgabe:
€42

NEXT LEVEL

Higher-Order Functions

$f(g(x))$

Das Konzept der **Higher-Order Function** stammt aus der funktionalen Programmierung.

Es handelt sich um eine Funktion (Methode), die eine oder mehrere der folgenden Eigenschaften hat:

1. Nimmt eine oder mehrere **Funktionen als Argument(e)**.
Beispiele: map, filter, reduce, sorted, forEach, orElseGet, orElseThrow
2. Gibt eine **Funktion als Ergebnis** zurück.
Beispiele: Collectors.*, Comparator.*, Stream.generate

NEXT LEVEL

Higher-Order Functions

$f(g(x))$

Beispiel:

`Comparator.comparing(Person::getAge)`

Die Methode `comparing` ist eine HOF. Ihr gebt ihr eine Funktion (hier die Methodenreferenz `Person::getAge`) und was bekommt ihr zurück?

Nicht das Alter, sondern eine *neue Funktion*, nämlich einen fertigen `Comparator`, der Personen nach Alter sortieren kann."

Oder:

`Collectors.toList()`

Das ist eine Methode, die eine Funktion zurückgibt, die der `collect`-Methode sagt, wie sie Elemente in einer Liste sammeln soll."

NEXT LEVEL

Higher-Order Functions

$f(g(x))$

Higher-Order Functions erlauben **flexiblen** und **wiederverwendbaren Code**.
Beispiel sind die Stream-Methoden.

In Java wurde diese **Flexibilität** durch die Einführung von **Lambdas** und der **Stream-API** ermöglicht. Durch diese lassen sich komplexe Operationen auf **deklarative** (beschreibende) Weise **ausdrücken**.



Beispiel:

```
Stream.of(1, 2, 3)
    .filter(number -> number == 2)
    .map(number -> number * number)
    .forEach(System.out::println); // Output: 4
```

Die Methoden filter, map und forEach sind alles Higher-Order Functions, da sie eine Function (Lambda) erhalten.

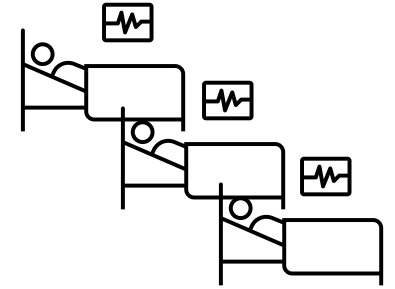
Higher-Order Functions: Functional Interfaces

Java verwendet für Higher-Order Functions generische [Functional Interfaces](#).
Erinnerung: Ein Functional Interface besteht aus genau einer abstrakten Methode.

Eine Auswahl vorhandener Functional Interfaces im JDK:

Functional Interface	Argument(e)	Ergebnis	Beispiele wo es verwendet wird
Runnable	0	void	Thread::new
Supplier<T>	0	<T>	Optional::orElseGet, Optional::orElseThrow
Consumer<T>	1 <T>	void	Stream::forEach, List::forEach
Predicate<T>	1 <T>	boolean	Stream::filter
Function<T, R>	1 <T>	<R>	Stream::map, List::replaceAll
UnaryOperator<T>	1 <T>	<T>	Stream::map, List::replaceAll
Comparator<T>	2 <T>	int	Stream::sorted, Stream::min, Stream::max
BiFunction<T, U, R>	2 <T, U>	<R>	Map::compute, Map::merge, Stream::reduce
BiConsumer<T, U>	2 <T, U>	void	Map::forEach

Higher-Order Functions: forEach Patient



```
record Patient(int id, int roomNumber) {}
```

```
class Hospital {
```

```
    LinkedList<Patient> patients = new LinkedList<>();
```

```
    void add(Patient patient) {  
        patients.addLast(patient);  
    }
```

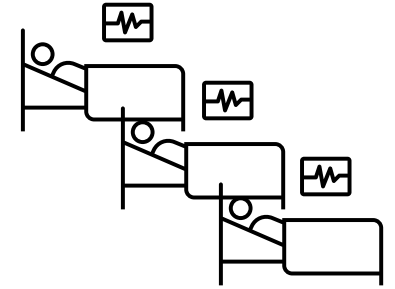
```
    static void main(String[] args) {  
        var hospital = new Hospital();  
        hospital.add(new Patient(1, 3));  
        hospital.add(new Patient(2, 5));  
        hospital.add(new Patient(3, 1));  
        hospital.forEach(patient -> System.out.println(patient.roomNumber()));  
    }
```

```
        void forEach(??? action) {  
            for (Patient patient : patients) {  
                action.accept(patient);  
            }  
        }  
    }  
}
```

Wie müsste das passende
Functional Interface definiert sein?



Higher-Order Functions: forEach Patient



```
record Patient(int id, int roomNumber) {}
```

```
class Hospital {
```

```
    LinkedList<Patient> patients = new LinkedList<>();
```

```
    void add(Patient patient) {  
        patients.addLast(patient);  
    }
```

```
    static void main(String[] args) {  
        var hospital = new Hospital();  
        hospital.add(new Patient(1, 3));  
        hospital.add(new Patient(2, 5));  
        hospital.add(new Patient(3, 1));  
        hospital.forEach(patient -> System.out.println(patient.roomNumber())); // Lambda hat 1 Parameter (hier: patient) und void als Rückgabewert.  
    }
```

Das **Functional Interface** `Consumer<T>` **bestimmt**,
welche **Signatur** das Lambda haben muss:
1 Parameter von Typ `T` und kein Rückgabewert (`void`)

```
@FunctionalInterface  
public interface Consumer<T> {  
    void accept(T t); // hier: accept(Patient patient);  
}
```

```
    void forEach(Consumer<Patient> action) {  
        for (Patient patient : patients) {  
            action.accept(patient);  
        }  
    }
```

Die Implementierung
wird als Lambda in
`Consumer<Patient> action`
gespeichert.

Erst hier wird das Lambda ausgeführt (**Lazy Evaluation**).

Der Methodenname **accept** stammt aus dem Functional Interface **Consumer<T>**.

Higher-Order Functions: forEachPatient

```
record Patient(int id, int roomNumber) {}
```

```
class Hospital {
```

```
    LinkedList<Patient> patients = new LinkedList<>();
```

```
    void add(Patient patient) {  
        patients.addLast(patient);  
    }
```

```
    static void main(String[] args) {  
        var hospital = new Hospital();  
        hospital.add(new Patient(1, 3));  
        hospital.add(new Patient(2, 5));  
        hospital.add(new Patient(3, 1));  
        hospital.forEach(patient -> System.out.println(patient.roomNumber()));  
    }
```

```
    void forEach(Consumer<Patient> action) {  
        for (Patient patient : patients) {  
            action.accept(patient);  
        }  
    }  
}
```



Unser Hospital bietet eine eigene forEach-Methode. Diese ermöglicht es, über alle Patienten zu iterieren und flexibel auf die Daten der Patienten zuzugreifen, die wir gerade benötigen.

*// Alternativ könnte man hier auch die id() anzeigen lassen.
// Oder statt System.out.println könnte man die Daten z. B. auch
// in eine Datei schreiben. Durch das Lambda ist die
// Implementation leicht durch Lambdas
// austauschbar (sogar zur Laufzeit!).*



Beispiel:

Higher-order Functions Übungen

Aufgabe 2: MovieDatabase

Pure Lambda



Lambdas sollen "pure" sein, d. h. **keine Seiteneffekte bzw. keinen Zustand** haben, damit sich Higher-Order Functions **vorhersagbar (deterministisch)** und **parallelisierbar** ausführen lassen:

1. Pure Lambda nimmt keine nach außen sichtbaren Veränderungen vor.
2. Pure Lambda hängt nicht von etwas ab, das sich von außen verändern kann.

Beispiel: `add(a, b)` ist pure. Das Ergebnis hängt nur von `a` und `b` ab."

Beispiel: `add(a)` das intern `a + globalVariable_b` rechnet, ist *nicht* pure, weil `globalVariable_b` sich ändern könnte.

Pure Lambda: Beispiel 1/2

Vermeide Lambdas, die etwas außerhalb manipulieren!



```
public static void main(String[] args) {  
    var result = new ArrayList<Integer>();  
    Stream.of(1, 2, 3)  
        .map(number -> result.add(number * 2)) // Not pure!  
        .toList(); // Änderung nach außen!!  
    System.out.println(result); // Wichtig für Multi-Threading.  
}
```

Ausgabe:

2
4
6

Lambdas in map, filter, sorted (also 'intermediate operations') müssen *immer* pure sein. Sie dürfen *nur* einen neuen Wert berechnen."

Pure Lambda: Beispiel 2/2

Vermeide Lambdas, die von außen manipuliert werden!



```
public static void main(String[] args) {
    int[] factor = {2}; // Alternativ würde das auch mit ArrayList, LinkedList usw. funktionieren.
    var stream = Stream.of(1, 2, 3)
        .map(number -> number * factor[0]);
    factor[0] = 0; // Not pure! Das Lambda wird von außen verändert!!
    stream.forEach(System.out::println); // Erst hier wird der Stream ausgewertet (Lazy Evaluation).
}
```

Ausgabe:

0
0
0



Pure Lambda: Closure

In Java bezeichnet ein Closure eine **anonyme Funktion** (aka Lambda), die auf **Variablen aus ihrem umgebenen Kontext zugreifen** kann, selbst wenn dieser **Kontext bereits beendet ist** (aufgrund von Lazy Evaluation).

Lambda-
Umgebung

```
IntConsumer addAndPrint() {  
    int baseValue = 100;  
    IntConsumer result = value -> { // Innerhalb der addAndPrint-Methode wird ein Lambda initialisiert.  
        int result = baseValue + value; // Das Lambda darf auf Variablen außerhalb seines eigenen Geltungsbereichs zugreifen.  
        System.out.println(result);  
    };  
    return result;  
}
```

```
public static void main(String[] args) {  
    addAndPrint().accept(50); // Output: 150  
}
```

↑
*Evaluation außerhalb von addAndPrint().
Lambda greift trotzdem auf baseValue zu.*

ABER: Externe Variablen, die innerhalb von **anonymen Funktionen** verwendet werden, müssen entweder **final** oder **"effectively final"** sein...



Pure Lambda: Effectively Final

In Java bezeichnet der Begriff **effectively final** eine **lokale Variable**, die nach ihrer Initialisierung **nicht mehr verändert (zugewiesen) wird**, auch wenn sie **nicht explizit mit dem final-Schlüsselwort deklariert wurde**.

```
class EffectivelyFinal {  
  
    public static void main(String[] args) {  
        int baseValue = 100; // effectively final variable  
  
        IntConsumer addAndPrint = value -> {  
            int result = baseValue + value;  
            System.out.println(result);  
        };  
  
        addAndPrint.accept(50); // Output: 150  
    }  
}
```

```
class EffectivelyNotFinal {  
  
    public static void main(String[] args) {  
        int baseValue = 100;  
  
        Consumer<Integer> addAndPrint = value -> {  
            int result = baseValue + value;  
            System.out.println(result);  
        };  
  
        baseValue = 42; // Cannot compile. Variable is no longer effectively final  
  
        addAndPrint.accept(50); // Output: none  
    }  
}
```



FYI: Die Variable darf sich nicht ändern, d. h., es ist **keine Neuzuweisung erlaubt**. Was erlaubt ist, den Objektzustand der Variablen zu ändern. Daher war das vorherige "Pure Lambda Beispiel 2/2" kompilierbar:

`int[] factor = {2}; ... lambda; ... factor[0] = 0; // effectively final, weil Zustandsänderung, aber keine Neuzuweisung. Nicht pure.`



Ausblick: Gatherer (Stream API)

Die Stream-API bietet einen umfangreichen, wenn auch festen Satz von intermediate Operationen wie map, filter, reduce, und sorted.

Diese Operationen sind für viele Aufgaben geeignet, aber der feste Satz von Zwischenoperationen bedeutet, dass sich einige komplexe Aufgaben nicht einfach als Stream ausdrücken lassen...

Beispiele: Sliding Windows, fold, scan

Gatherer erlauben es, **eigene intermediate Methoden** für Streams zu schreiben. 🤖

Siehe: <https://openjdk.org/jeps/461> (ab JDK 24).

Callbacks



Callbacks



Ein **Callback** ist eine bestimmte Art von **Higher-Order Function**.

Hierbei wird **eine Funktion (Lambda) als Argument** an eine **andere Funktion übergeben**, welche die übergebene Funktion später aufruft (Lazy Evaluation).

Callbacks ermöglichen es, auf **Ereignisse zu reagieren**, sobald diese eintreten.

Beispiele:

- Button in GUI-Frameworks, um Ereignisse wie `onClick(lambda)` zu reagieren.
- Abschluss eines Netzwerktransfers `onDownloadCompleted(lambda)`.
- Methoden mit `@BeforeEach` und `@AfterEach` in Unit-Tests sind eine spezielle Form von Callbacks (mittels Annotationen realisiert statt Lambdas).

Begriffe: Callback, Event, Hook

Callback: Allgemeiner Begriff für eine Funktion, die als Parameter übergeben und zu einem späteren Zeitpunkt aufgerufen wird.



Event: Bezeichnet das Reagieren auf ein Ereignis, sobald dieses eintritt, z. B. eine Benutzeraktion oder ein Systemereignis. Der Programmablauf kann durch ein Event unterbrochen oder parallel (asynchron) verarbeitet werden. Events lassen sich mittels Callbacks realisieren.



Hook: [Hooks](#) sind spezielle Callbacks, die oft von Frameworks bereitgestellt werden. Durch Hooks erhält man die Möglichkeit, eigenen Code in die fremden Funktionen eines Frameworks "einzuschleusen".



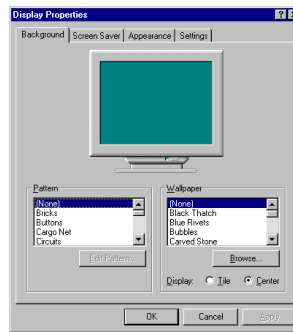
Callback: Beispiel 1

```
class DownloadManager {  
  
    void downloadFile(String url, Consumer<String> callback) {  
        System.out.println("Datei wird heruntergeladen...");  
        // ... downloading file from url ...  
        String message = "Download abgeschlossen!";  
        callback.accept(message); // evaluate callback  
    }  
}  
  
class DownloadSimulator {  
  
    public static void main(String[] args) {  
        DownloadManager downloader = new DownloadManager();  
        downloader.downloadFile(  
            "https://de.wikipedia.org/wiki/R%C3%BCckrufffunktion#/media/Datei:ProgramExecCallbackSimple-de.png",  
            message -> System.out.println(message) // provide a callback method that is called later.  
        );  
    }  
}
```

Ausgabe:

```
Datei wird heruntergeladen...  
Download abgeschlossen!
```

Callback: Beispiel 2 (Swing)



```
class DisplayPropertiesView extends JView {  
  
    private final Properties properties = new Properties();  
  
    private final JButton okButton = new JButton();  
    private final JButton cancelButton = new JButton();  
  
    DisplayPropertiesView() {  
        initializeWidgets();  
    }  
  
    private void initializeWidgets() {  
        cancelButton.onClicked(() -> properties.discard());  
  
        okButton.onClicked(() -> {  
            properties.save();  
            Popup.show("Properties saved");  
        });  
    }  
}
```

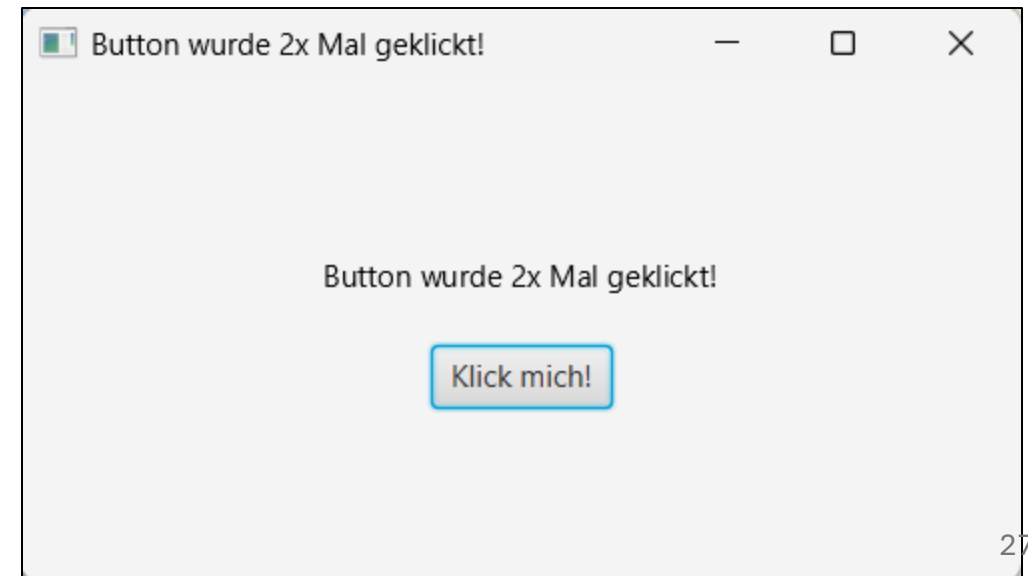
```
class JButton { // this class is part of a GUI framework  
  
    private Runnable action; // stored action  
  
    void onClicked(Runnable action) {  
        this.action = action; // store action  
    }  
  
    // click is called by GUI framework when user clicks this button  
    void click() {  
        action.run(); // evaluate stored action  
    }  
}
```

```
@FunctionalInterface  
public interface Runnable {  
  
    void run();  
}
```

Callback: Beispiel 3 (JavaFX)

```
public class HelloApplication extends Application {  
    private int clickCounter = 0;  
  
    @Override  
    public void start(Stage stage) throws IOException {  
        Parent root = loadFXMLFile("hello-view.fxml");  
  
        // Set callback for button  
        Button clickMeButton = getButtonWidget(root, "#clickMeButton");  
        clickMeButton.setOnAction(event -> {  
            String message = "Button wurde %sx Mal geklickt!".formatted(++clickCounter);  
  
            System.out.println(message);  
  
            Text welcomeText = getTextWidget(root, "#welcomeText");  
            welcomeText.setText(message);  
  
            stage.setTitle(message);  
        });  
  
        // Setup scene  
        VBox layout = (VBox) root.lookup("#container");  
        Scene scene = new Scene(layout, 400, 200);  
        stage.setTitle("JavaFX Callback Demo");  
        stage.setScene(scene);  
        stage.show();  
    }  
  
    // ...  
    public static void main(String[] args) { launch(); }  
}
```

```
<?xml version="1.0" encoding="UTF-8"?>  
  
<?import javafx.geometry.Insets?>  
<?import javafx.scene.layout.VBox?>  
<?import javafx.scene.control.Button?>  
<?import javafx.scene.text.Text?>  
  
<VBox fx:id="container" alignment="CENTER" spacing="20.0" xmlns:fx="http://javafx.com/fxml">  
    <padding>  
        <Insets bottom="20.0" left="20.0" right="20.0" top="20.0"/>  
    </padding>  
  
    <Text fx:id="welcomeText" text="Klick den Button!"/>  
    <Button fx:id="clickMeButton" text="Klick mich!"/>  
</VBox>
```



Callback: Beispiel 4 (Multiple Callbacks)



```
class 🚫 NuclearPoweredWaffleMaker {  
  
    private static final int ONE_SECOND = 1_000; // FYI: _ in numbers can be used to improve readability.  
  
    private final Collection<OnCompleteCallback> callbacks = new LinkedList<>(); // stores callbacks.  
  
    void subscribeOnComplete(OnCompleteCallback callback) {  
        callbacks.add(callback); // store additional callback  
    }  
  
    void bake() throws InterruptedException {  
        System.out.println("Baking...");  
        Thread.sleep(ONE_SECOND);  
        callbacks.forEach(callback -> callback.notify("Waffle ready!")); // Evaluate ALL stored callbacks.  
    }  
  
    public static void main(String[] args) throws InterruptedException {  
        var waffleMaker = new NuclearPoweredWaffleMaker();  
  
        waffleMaker.subscribeOnComplete(message -> System.out.println(message)); // Callback #1  
        waffleMaker.subscribeOnComplete(message -> new GreenLed().blink()); // Callback #2 – FYI: message could be replaced with _  
        waffleMaker.subscribeOnComplete(_ -> new Buzzer().beep()); // Callback #3 – FYI: _ indicates a parameter that is not used.  
  
        waffleMaker.bake();  
    }  
}
```

```
@FunctionalInterface  
interface OnCompleteCallback {  
    void notify(String message);  
}
```

Hier wird ein eigenes Interface verwendet statt `Consumer<T>`.

Grund: Der Name des Interfaces hat mehr Semantik, d. h., er deutet bereits darauf hin, wozu das Interface verwendet wird. `Consumer<String>` könnte hier aber auch verwendet werden.

Callback: Unit Test

Callbacks müssen oftmals **anders getestet** werden, da die eigentliche Methode, die getestet werden soll, oft **void zurückgibt** (muss aber nicht).

Im Testfall wird zunächst eine eigene Implementation eines Callbacks mitsamt der **assert-Methoden** übergeben (assert).

Erst anschließend wird die **zu testende Methode aufgerufen** (act).

Act und **Assert** werden in diesem Fall **in anderer Reihenfolge** implementiert. 🤪

```
@Test
void bakePublishesOnCompleteCallback() throws InterruptedException {
    // Arrange
    var waffleMaker = new NuclearPoweredWaffleMaker();

    // Assert (im Callback)
    waffleMaker.subscribeOnComplete(message -> { // subscribe a custom callback for this testcase
        assertEquals("Waffle ready!", message); // assert is within the test callback
    });

    // Act
    waffleMaker.bake(); // method to be tested: must be called after subscribe!
}
```



Callbacks: Wozu?



In dem vorherigem Beispiel ☢️ NuclearPoweredWaffleMaker könnte man die Callbacks doch einfach direkt in der bake-Methode implementieren? Wozu Callbacks? #WTF

Separation of Concerns: Callbacks ermöglichen es, den Backvorgang von den nachgelagerten Aktionen zu trennen. Die bake-Methode kümmert sich nur um das Backen und nicht darum, was danach passiert. Dies führt zu saubererem und modularerem Code.

Wiederverwendbarkeit und Flexibilität: Durch die Verwendung von Callbacks lassen sich beliebige Aktionen nach dem Backen ausführen, ohne die bake-Methode ändern zu müssen. Dies macht den Code wiederverwendbarer und flexibler.

Erweiterbarkeit: Neue Funktionen können hinzugefügt werden, indem neue Callbacks registriert werden, anstatt die bestehende Logik zu ändern und testen zu müssen.

Event-gesteuerte Programmierung: Callbacks sind ein wesentlicher Bestandteil der asynchronen Programmierung und ermöglichen es, auf Ereignisse zu reagieren, sobald sie eintreten, ohne den Hauptfluss des Programms zu blockieren.

Beispiel:

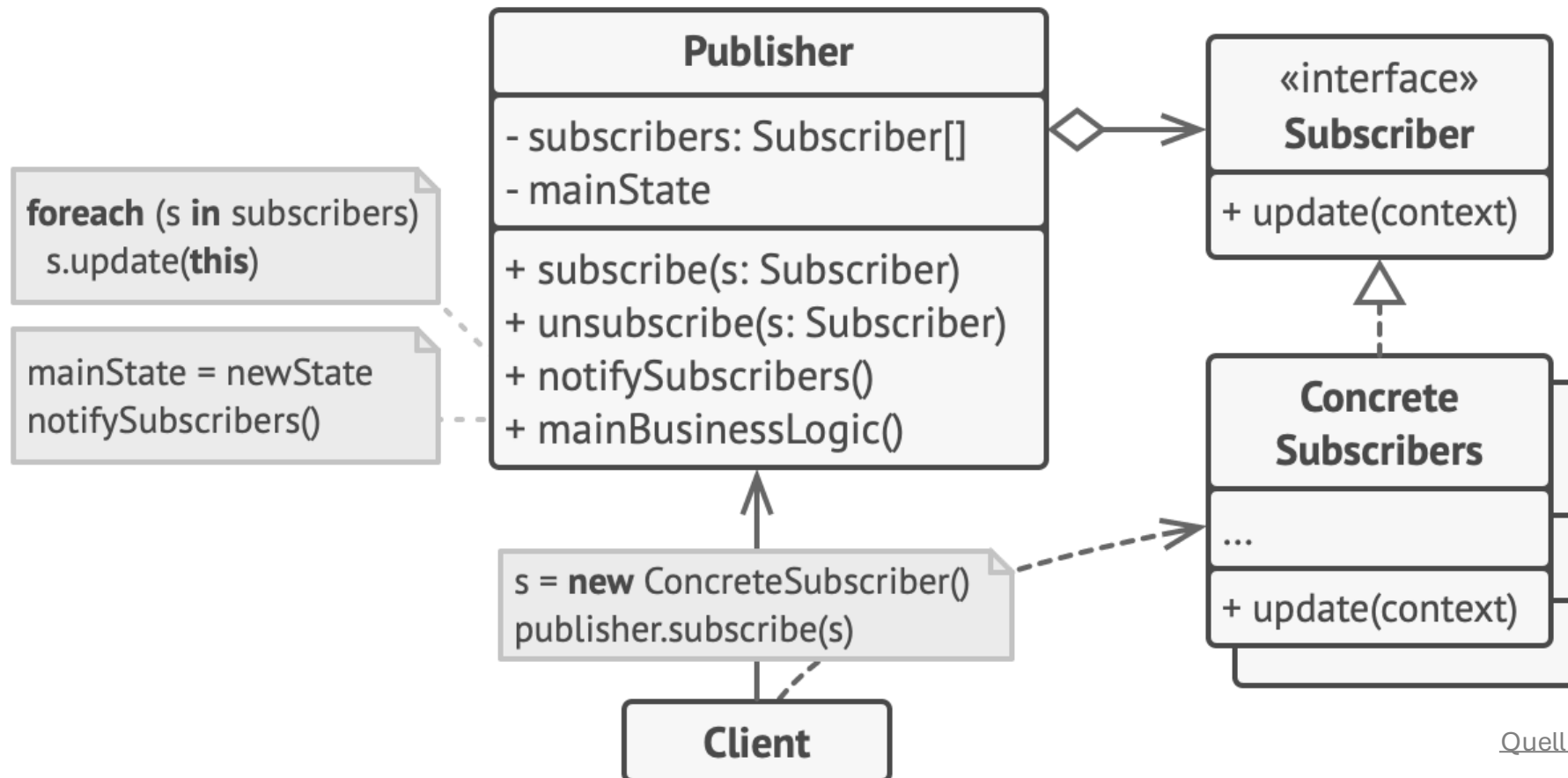
Callback Übungen

Aufgabe 4: Big Brother

Observer Pattern



Ein **Callback** ist eine spezielle Implementierung des **Observer Patterns**.



Observer: Beispiel

```
class Counter {  
  
    private final List<Subscriber> subscribers = new LinkedList<>();  
    private int count = 0;  
  
    void subscribe(Subscriber subscriber) { subscribers.add(subscriber); }  
    void unsubscribe(Subscriber subscriber) { subscribers.remove(subscriber); }  
    void notifySubscribers() { subscribers.forEach(subscriber -> subscriber.update(count)); }  
  
    void increment() {  
        count++;  
        notifySubscribers();  
    }  
  
    public static void main(String[] args) {  
        Publisher publisher = new Publisher();  
  
        Subscriber callback = count -> System.out.println(count);  
  
        publisher.subscribe(callback);  
        publisher.increment();  
        publisher.increment();  
  
        publisher.unsubscribe(callback);  
        publisher.increment();  
    }  
}
```

```
interface Subscriber {  
    void update(int context);  
}
```

Ausgabe:

1
2

Observer: Beispiel (JUnit)

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CounterTest {

    private Counter counter = new Publisher();

    @Test
    void subscribe() {
        counter.subscribe(count -> assertEquals(1, count));
        counter.increment();
    }

    @Test
    void unsubscribe() {
        int[] modifiableCounter = {0}; // helper variable to count the number of notifySubscribers()-calls
        Subscriber subscriber = _ -> modifiableCounter[0]++; // not pure lambda, but in Unit Test okay...

        // Act
        counter.subscribe(subscriber);
        counter.unsubscribe(subscriber);
        counter.increment();
        counter.subscribe(subscriber);
        counter.increment();

        assertEquals(1, modifiableCounter[0]++);
    }
}
```

Lernziele HFO

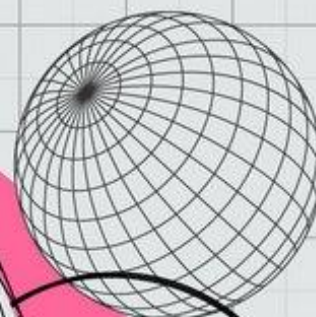
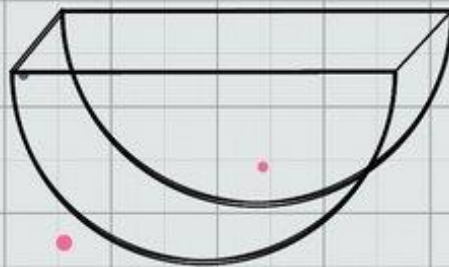
- **Standard-Interfaces erkennen und nutzen**
- **Eigene HOFs implementieren (die Lambdas *empfangen*)**
- **Übergebene Logik anwenden**
- **Eigene FunctionalInterfaces definieren**
- **Datenflüsse verstehen (map/filter)**

Lernziele Callbacks

- **Das Observer Pattern erkennen**
- **Den Publisher implementieren**
- **Das Event auslösen**
- **Semantische Interfaces nutzen**
- **Callback-Code testen**

Tag der offenen Tür im *ligeti zentrum*

22. November 2025
11 bis 18 Uhr



ligeti
zentrum



Innovation zum Anfassen

